Spike: 6
Title: Emergent Group Behaviour
Author: Pattarapol Tantechasa, 103883220

Goals / deliverables:

Expected Outcomes (Baseline Task)

1. A working simulation demonstrating distinct modes of emergent group behaviour.
2. Implementation of cohesion, separation, alignment, and wandering steering behaviours.
3. A functional weighted-sum system combining all steering behaviours.
4. A user interface/input system allowing real-time adjustment and display of parameters for each steering force.
5. A comprehensive Spike Outcome Report (PDF format) and working code with key instructions provided in the README.md.

Expected Outcomes (from extensions)

1. Code implementation of rigid body collision constraints and a write-up in the Spike Report regarding the required parameter adjustments to accommodate this.
2. A visible predator agent in the simulation that successfully repels the standard flock.
3. Static environmental obstacles (walls) with functional "feeler" logic preventing clipping, including a documented configuration that induces continuous circling.
4. Multiple distinct classes of agents with unique steering configurations, demonstrating complex inter-group dynamics (e.g., fleeing, chasing, or isolating).

Technologies, Tools, and Resources used:

- Claude: brainstorm ideas, explain code and debugs.
- Visual Studio Code IDE
- Conda: Python environment manager

Tasks undertaken:

1. Download and Install **Visual Studio Code**
2. Install **Python** and **Pyglet** on your Machine
3. Open the main project downloaded
4. Open terminal and run cd command : **cd Portfolio-Tasks/Portfolio-Task-14-Spike-Emergent-Group-Behaviour**
5. For baseline version : **cd emergent-group-baseline**
6. For extended version : **cd emergent-group-extensions**
7. Run : **python main.py**

Design Phase: Emergent Group Behaviour

Overview

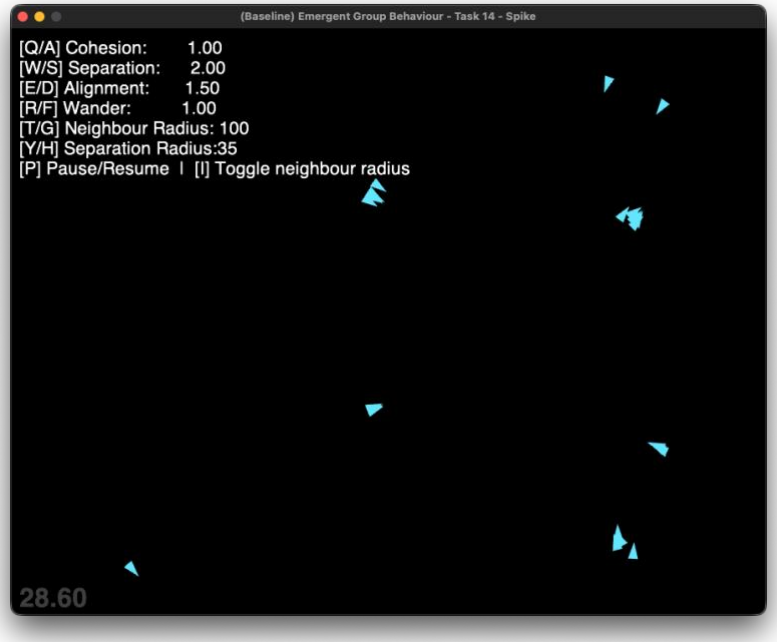
The goal of this spike was to explore how three foundational steering forces (cohesion, separation, and alignment) can be combined with wandering to produce emergent group behaviours such as flocking and schooling. Rather than scripting any single agent's movement, I built a system where each agent only responds to its immediate neighbours, and the collective behaviour emerges from those local rules alone. I used the existing steering codebase from Task 13 (wander, seek, and the physics integration loop already implemented) as the foundation and built the group system on top of it.

Agent Setup

I extended the existing Agent class to support multiple simultaneous instances. Each agent is spawned with a randomised position and heading across the window, and all share the same World reference so they can query each other's positions and headings each frame. For the baseline I spawned 25 agents, all rendered as small cyan triangles.

Each agent tracks:

- **pos** — current world position
- **vel** — current velocity
- **heading** — normalised direction vector (derived from velocity)
- **wander_target** — local jitter target used by the wander behaviour



Neighbourhood Detection

Each frame, every agent calls `get_neighbours()` which iterates all other agents and returns those within `neighbour_radius`. This radius controls how large each agent's "awareness zone" is. I gathered the average position and average heading of the returned neighbours, which are the two values needed to compute cohesion and alignment respectively. The neighbour radius is adjustable at runtime so I could observe how larger or smaller awareness zones changed the emergent behaviour.

Core Behaviours

I implemented four steering behaviours in the Agent class:

Cohesion steers the agent toward the average position of its neighbours. The target is the centroid of all detected neighbours, and the force is computed using a seek towards that point. This keeps the group together over time.

Separation steers the agent away from neighbours that are closer than a separate, smaller `sep_radius`. The repulsion force is weighted by inverse distance, the closer the neighbour, the stronger the push which preventing agents from crowding into the same point.

Alignment matches the agent's heading to the average heading of its neighbours. I computed the average of all neighbour heading vectors, normalised the result, then used it as a desired velocity direction scaled to `max_speed`. The resulting force is `desired − current_velocity`, which is the same velocity-space approach used by seek and flee, ensuring alignment is comparable in magnitude to the other forces.

Wandering uses the existing projected jitter circle from Task 13. A small random offset is added to a target point on a circle projected ahead of the agent each frame, producing smooth unpredictable movement. This prevents the flock from freezing into a stationary cluster when all other forces balance out.

Weighted-Sum Blending

I combined all four behaviours inside `agent.calculate()` using a weighted sum:

```
total_force = (wander    x w_wander)
            + (cohesion  x w_cohesion)
            + (separation x w_separation)
            + (alignment  x w_alignment)
```

Each weight is a float stored on the World object so all agents share the same global tuning values. The resultant force is then truncated to max_force before being applied through the standard physics integration loop (F = ma).

Interactive Parameter Tuning

I added real-time keyboard controls and on-screen labels so all weights and radii can be adjusted while the simulation runs. The displayed parameters are:

Parameter	Keys	Default
Cohesion weight	Q / A	1.0
Separation weight	W / S	2.0
Alignment weight	E / D	1.5
Wander weight	R / F	1.0
Neighbour radius	T / G	100 px
Separation radius	Y / H	35 px

Pressing I toggles a grey neighbourhood radius ring around every agent, making the detection zone visible.

Observed Emergent Modes

By adjusting the weights, I identified four distinct emergent modes:

Tight school — high cohesion and alignment with low wander produces a tightly packed group all moving in the same direction, resembling a school of fish.

Loose swarm — low cohesion with moderate separation and high wander produces a loose cloud of agents drifting independently but staying loosely associated.

Scattered drift — zeroing both cohesion and alignment while keeping wander active causes the flock to dissolve. Each agent wanders independently with no group structure.

Repulsion field — very high separation with low cohesion causes agents to actively spread out and fill the available space, each maintaining its own territory.

Extension 1: Non-Overlapping Agents

In the baseline, agents are treated as point masses with no physical size. Under high cohesion or high separation settings, multiple agents can occupy the same pixel, which looks unrealistic. To address this, I added a rigid body collision resolution pass that runs after all agents update their positions each frame.

For every pair of agents, I compute the distance between them. If the distance is less than the sum of their radii (agent_radius per agent), I push both agents apart by equal amounts along the separation axis, so neither agent moves more than the other:

```
overlap = (min_dist - dist) * 0.5
push = diff.get_normalised() * overlap
a.pos -= push
b.pos += push
```

This is a hard position constraint applied after the steering physics, meaning it always enforces separation regardless of what the weighted-sum produces. The outcome is that even under extreme cohesion settings, agents arrange themselves into a compact cluster with visible gaps between individuals rather than stacking invisibly.

The effect on parameter tuning was notable: without non-overlapping constraints, high separation weight was necessary to prevent visual stacking. With the constraint active, separation weight could be reduced because the rigid body system handled close-contact cases, and the separation steering force only needed to manage intermediate distances.

Extension 2: Predator Avoidance

I added three predator agents (large red triangles) that operate independently from the flock. Each predator uses a simple seek behaviour targeting the nearest prey agent every frame, with a slightly higher max speed than prey to create genuine threat pressure.

Prey agents have an additional `flee_predator()` steering force that activates when any predator enters a 160px panic radius. The force direction is away from the predator, weighted by inverse distance so closer predators produce a stronger response:

```
away = self.pos - pred.pos
away.normalise()
steer += away * (PANIC_RADIUS / max(dist, 0.001))
```

The flee force is added to the weighted sum alongside cohesion, separation, alignment, and wander. The runtime U / J keys adjust its weight so I could compare the flock's response with and without predator awareness active.

The observable outcome is that a calm, organised flock instantly fractures when a red predator enters range. Agents closest to the predator flee first, pulling the rest of the flock apart through the cohesion force as the group tries to stay together while some members flee. Reducing the flee weight to zero causes prey to completely ignore predators and maintain their flock structure regardless of proximity, which visually demonstrates the effect of that single parameter.

Extension 3: Wall Avoidance

I added a static horizontal wall segment across the centre of the world and implemented a three-feeler raycasting system to steer agents around it.

Each frame, every agent projects three feeler rays from its current position in its direction of travel: one straight ahead (120px), one angled 35° left (78px), and one angled 35° right (78px). For each feeler, I test whether the ray intersects the wall segment using the standard 2D parametric line intersection formula, checking that both the feeler parameter t and the wall parameter u are within [0, 1]:

```
denom = r.x * s.y - r.y * s.x
t = (w.x * s.y - w.y * s.x) / denom
u = (w.x * r.y - w.y * r.x) / denom
if 0.0 <= t <= 1.0 and 0.0 <= u <= 1.0:
    hit_pt = self.pos + r * t
```

When an intersection is found, I calculate the outward normal of the wall at that point and compute a velocity-space steering force: `desired = normal * max_speed`, then `force = (desired - velocity) * (feeler_length / distance_to_hit)`. This matches the scale of cohesion and seek forces so the wall weight can be tuned fairly against the other behaviours. The closer the intersection point, the larger the scaling factor and the sharper the turn.

I added a secondary hard correction pass after each update. If any agent penetrates within 10px of the wall, its position is ejected to the clearance boundary and the velocity component directed into the wall is removed. This prevents high-speed agents from tunnelling through the wall when the feeler steering force alone is insufficient.

The outcome is agents curving around the ends of the wall segment rather than passing through it. Lowering the wall avoidance weight below 0.5 visually demonstrates the feeler system's contribution — agents start clipping through the wall as the steering force loses out to cohesion.

Extension 4: Factional Behaviours

I created three distinct agent factions — Prey, Predator, and Scavenger — each with a unique steering configuration and cross-faction dynamics, producing a simple predator-prey-scavenger food chain.

Each agent carries a `faction` integer constant. The `get_neighbours()` method filters by faction, so cohesion, separation, and alignment only consider agents of the same type. Cross-faction interactions are handled by dedicated methods called separately in each faction's `calculate()` branch.

Prey (cyan, 20 agents) run the standard four-behaviour weighted sum plus the `flee_predator()` force from Extension 2. They flock tightly among themselves and scatter when a predator enters range.

Predators (red, 3 agents) skip the flocking system entirely. Their `calculate()` returns only wander (at a low weight for baseline movement) plus `seek_nearest_prey()`, which finds the closest prey agent each

frame and seeks toward it. They are heavier and larger than prey, giving them more momentum and visual presence.

Scavengers (yellow, 7 agents) run the same four-behaviour flock system as prey but among themselves, plus a `follow_predators()` force. This force computes the average position of all predator agents and seeks toward it, but only activates when the scavenger is more than 130px from that average. Inside that distance, the force returns zero, so scavengers cluster around predators at a safe trailing distance rather than pursuing indefinitely.

The emergent outcome is a layered dynamic: predators hunt prey, prey scatter and regroup, scavengers drift toward wherever predators are, forming an observable chain of behaviours from three simple rules. Zeroing the flee predator weight (J) removes prey's awareness of predators entirely, demonstrating that the chain only holds when each layer of interaction is active.

Conclusion

I built a working emergent group behaviour simulation where complex, lifelike movement arises entirely from each agent evaluating a small set of local rules. The baseline demonstrated that cohesion, separation, and alignment alone are sufficient to produce qualitatively different emergent modes — from tight schooling to scattered drifting — purely through weight adjustment, with no scripted group-level behaviour anywhere in the code.

The extensions progressively added complexity: rigid body constraints ensured physical realism regardless of weight settings; predator avoidance demonstrated how a single additional force can fracture and reform a flock dynamically; feeler-based wall avoidance introduced environmental awareness without any global pathfinding; and factional behaviours showed that the same local-rule architecture scales to multi-species ecosystems where each type's interaction produces a coherent emergent food chain.

The key insight from this spike is that emergent group intelligence does not require coordination or communication between agents. It requires only that each agent responds to the same simple local signals — neighbour position, neighbour heading, proximity threats — and that those signals are combined with well-chosen relative weights.